天猫新品团队AI编码实战指南（下）

原创 天猫新品营销技术 大淘宝技术 2026年5月8日 01:07 浙江 308人



本文是关于 AI 辅助编码的全面实战指南，基于天猫新品团队的实践经验，从问题本质到解决方案，从理论框架到实战案例，系统性地介绍如何让 AI 更好地完成大部分需求。

本文分上下两篇，天猫新品营销技术团队AI编码实战指南（上）包含：

1. 现状与问题诊断 – 深入剖析 AI 生码的四大痛点（写不对、写不好、写不了、改不动），并从项目知识、用户输入、任务复杂度、自检机制、模型能力等五个维度提供针对性解法。
2. 方法论与优化思路 – 提出"最大化复用、自然语言第一、二八定律"三大核心思想，并沿着"前置准备→开发前→开发中→完成后"的全流程，给出每个节点的可落地优化手段。
3. 分场景实战案例 – 根据验收标准和代码质量要求，将需求分为"需求驱动型"和"工程主导型"两类，通过小二端列表页和C端复杂业务的完整案例，展示不同场景下的最佳实践。

本篇包含：

4. 团队建设经验 – 分享新品团队在小二端（后端全栈化）和C端（视图分离、知识库建设、工作流沉淀）两个方向的探索，包括工具建设、文档沉淀、知识库方案等具体落地内容。
5. 实用技巧集锦 – 涵盖 UI 重构、复杂 Prompt 构建、数据转换、多方案选优、文档生成等常见应用场景，以及严厉语气、合理质疑等提升准确度的技巧。



背景

为了顺应 AI 时代的潮流，我们团队开始了后端向前，前端向后的全栈化转型运动。转型前期，考虑将一些小二工作台与研发工具的开发交由后端（ 主要是交付分割线下面的部分，即前面所提到的需求驱动型 ），而前端则是承担一些 C 端 solution 的编写。因此，我们从这两个角度都进行了一定程度的 AI 生码探索，也沉淀了一些相应的开发经验与内容。

严苛程度	典型场景	错误容忍程度	交互问题容忍程度	代码要求	视觉还原要求
最为严苛	C 端频道	0 容忍	明显则 0 容忍	高	高
较为严苛	B 端商家平台	0 容忍	特别明显则 0 容忍	中	中
交付分割线					
普通严苛	小二端工作台	0 容忍	影响主链路则 0 容忍	低	低
低严苛	研发自用工具与平台	一定程度容忍	没有 hack 绕过的方案则 0 容忍	无	无
不严苛	研发 DEMO	能意会则能容忍	怎么都能容忍	无	无

关于我们在 AI 生码的思路：

作为业务团队，在AI基建不断迭代背景下，我们的重点应该是沉淀自己团队的工作流与 AI 资产，从而在有颠覆性新产品出现时可以快速接入，且这部分是其他AI工具无法为代劳，只能由我们自己进行建设与沉淀的。

基于此，我们在这一块的最主要思想就是通过最大化复用来提高整体效能，包括但不限于代码、知识、工作流、工具的复用，后面的内容也主要会围绕这一条主线来展开。

小二端 - AI 主导的对话生码

这部分需求的主要特点是：没有视觉还原度要求，实现形式较为自由，所以自然地避开了占较多开发时间的精调环节；且页面间独立性强，项目耦合度低，适合使用 AI 编码完成全部需求。

在这类场景中，后端同学主要通过AI完成全部需求实现，基本不会去介入实际代码。

- 初期 – 统一生成方案

首先最要紧的就是，面向这么多前端练习生，以及他们各自使用的不同编码工具，怎么让他们尽可能地产出风格一致的代码，以及视觉风格贴近小二端视觉规范的页面；同时，还需要保证一定的AI迭代成功率。对此，我们最开始主要主要提供了这些辅助手段。

- 小二端开发培训

- 基础的代码实现规范
  - 基于小二端的特性，采用页面间完全解耦的结构，减少复杂度
  - 通过一些代码约束提高代码生成质量，从而提高迭代生码的成功率
- 针对一些高频页面提供代码模版，进一步约束产出结果

- 中期 – 辅助补齐前端经验短板

在上面的基础上运行过一段时间以后，我们发现了一些后端同学由于前端经验不足而碰到的高频问题：

需求描述不准确

在实行初期，碰到最主要的问题就是，后端同学因为对前端一些的专业术语没有概念，没有办法精准地描述出自己脑海中的那个需求，导致频繁返工，AI 始终无法按需求完成内容。

首先，我们在后台新增了「AI案例实践中心」，将B端常见的页面都归纳到了标准化的 prompt 模版，方便快速查找案例与实现。



同时，我们还基于此提供了对应的MCP速查工具「天猫新品业务编码助手」，后端同学通过可以直接通过工具对话来获取自己想要的页面模版，输入自己的需求后，即可得到一份标准化的页面 PRD 来提供给 AI 进行页面生成。



还有一种情况，就是后端同学已经在其他地方找到了参考的页面，只是不知道该如何对自己的 AI 进行需求描述。我们也提供了基于 Gemini 3.0 的多模态能力特调的图片生成提示词工具，尽可能保证系统化地描述页面，相较于直接传图给编程工具，转换后的结构化描述词有更好的页面还原效果。



垂类场景 / 复杂问题没有经验

对于初入前端开发的后端同学，还有一些经验带来的问题，比如：

- 对一些常见场景 / 垂类场景的实现没有概念，没有经验，容易被AI的临时方案带跑；
- 对一些坑没有经验，开发容易陷入死循环。

借着这个机会，我们将 AI 生码省下来的时间提出来一部分用于生码沉淀，完成需求的过程中详细记录实现过程与踩坑心得。案例分为新增型需求和更新型需求，每个需求会详细描述清楚需求背景、实践步骤与功能的自测，最后还会有过程中的错误提示。整体上作为典型实践案例足以面向日常的需求开发。



- 后期 – 需要更简单、无感、且一致的方案

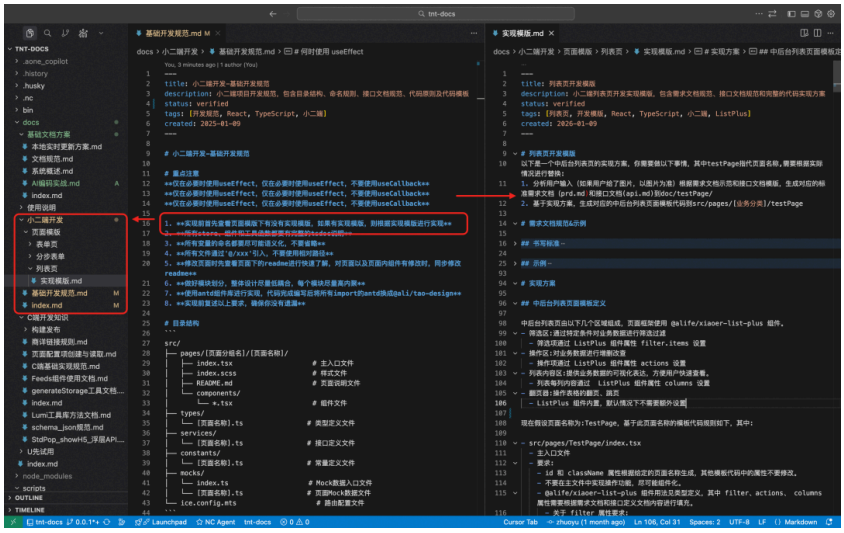
基于前面的工作，我们的后端全栈化已经平稳运行了一段时间，基于相关同学的体验反馈，我们又收集上来了以下几个问题：

- 大部分人还是更倾向于使用基础的 Chat 对话模式来完成需求，不想使用类Spec的规划模式，也不想跳出编辑器进行额外的操作（甚至不想配mcp）；
- 团队内业务复杂，相关仓库也很多，一些文档以及基础规范的改动无法及时同步，希望有实时更新的统一方案。

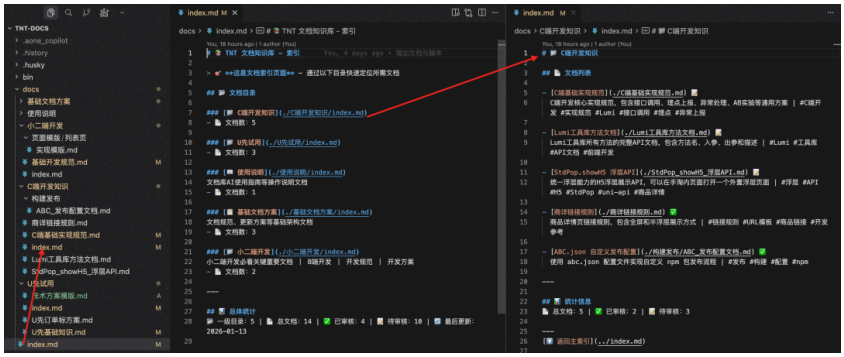
所以，我们为小二端开发提供了一个轻量级的团队知识库（其实更多用在C端开发），以类Skill的形式封装了小二端开发的规范与代码模版，实现了无视开发工具，简单易用的公共知识库，通过公共知识库进行小二端 AI 开发的无感辅助。

知识库直接使用 git 仓库进行管理，npm 包作为资源承载与版本管理的工具：

- 以类Skill的形式封装了小二端开发的规范与代码模版，完成了团队内 AI 开发方案的统一收口；



- 使用文件链接进行文档的多级管理与索引，渐进式披露，不会过度占用 token



- AI 可以直接通过 curl 命令读取 npm 包下的 cdn 资源，从中获取必要知识，不依赖具体编码工具，且接入简单，直接配置在编辑器Rules即可；



- 甚至还可以直接借用CodeWiki进行知识库问答



这时，就天然地实现了一个及时更新、不依赖编码工具、支持问答、使用无感的统一知识源，完成了团队内基础开发方案的统一收口。

- 未来展望

目前还剩下的一些问题需要进一步思考解法：

- 精细化微调无法达到效果
  - 后端同学上手前端以后，也有了精调页面细节的热情，但是只用 AI 对话的场景下很难完成这些精细化的页面操作，有没有什么办法去 cover 这部分的编码需求？
- 开放式场景如何保证尽量统一
  - 虽然对于枚举过的页面，现有的约束条件可以一定程度上约束产出内容，但是对于未枚举，或者无法枚举的页面，产出的页面就随着使用者或者编码工具的不同而开始天差地别，（有时候甚至可以通过页面风格判断是谁，用什么工具写的页面）有什么办法可以让这些部分也尽可能地有一个较为一致的视觉表现？

C端 - 全栈开发模式下的 AI 辅助

这部分主要有如下几个主要建设方向：

- 这类需求代码质量与验收标准较高，且业务隐含逻辑多，没有办法直接通过AI完成全部内容，怎样提高AI在其中的参与度?
- 整体开发链路长，涉及仓库多(包括前后端、组件库与工具库)，怎样进行有效串联，提高整体效率?
- 对于开发链路上的一些高频高耗时的节点，怎样进行分析与优化?

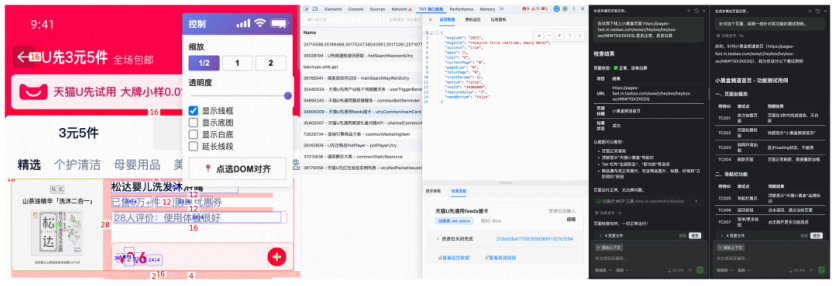
大部分内容前面C端实战部分已经提到过，这里不再赘述，主要总结下我们的主要 Action。

- 主要 Action

- 代码结构与公共组件的设计，提高生码效率与代码可控度
- 多类高频开发流程的标准化 workflow 的沉淀
- 基础团队知识库，提供统一的读取方案与快速的知识查询能力



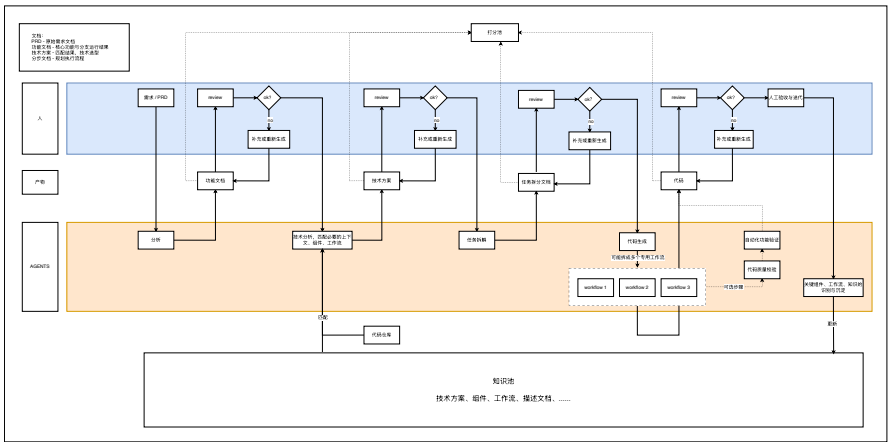
- 让 AI 基于知识库内容产出技术方案，并进行人工审核（后期考虑提供专用的技术方案 Agent，并进行技术方案的打分评测）
- 开发流程中的工具辅助（走查辅助、接口查询、页面检查MCP）



- 未来方向

- 完善知识库的 生成与自动更新 方案，弱化人工参与度，提高资产沉淀效率
- 建设更加 AI 友好的代码架构，提高整体的生码效果
- 进一步提高整条链路的 AI 串联度，完成整体提效

- 开发流程的串联
- 现有工具的串联



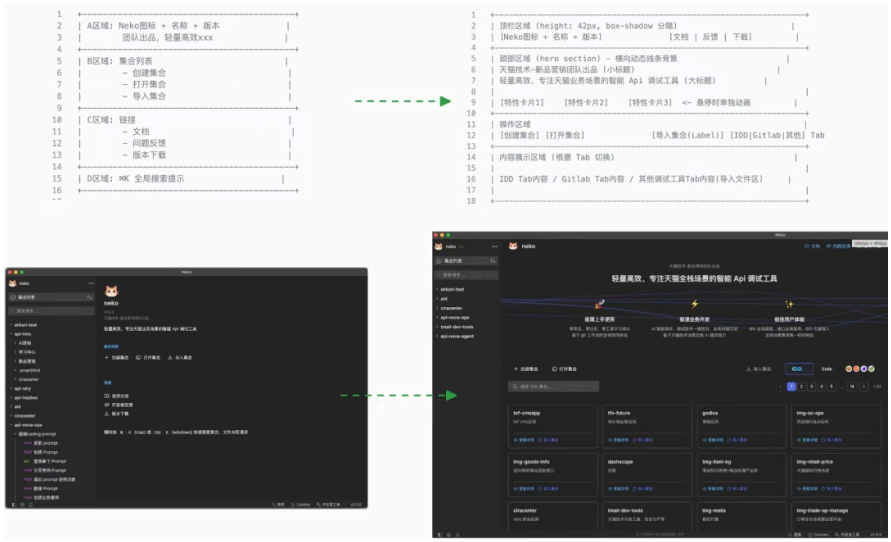
理想化的C端纯 AI Coding 流程

02 活用 AI，掌握编码小技巧

还能实现什么功能？

- UI 布局重构

- 让 AI 解释旧布局，对旧页面结构进行区块划分与标记
- 构思新布局，描述清楚旧内容的迁移方向，并增加新的功能



让 AI 理解布局，让 AI 瞎子也能看得见

- 复杂 prompt 构建

- 案例【neko 智能参数调试面板功能】我希望用 AI 来生成一个调试界面，最终实现如下诉求：
  - 理解 http 请求 get/post 的参数/正文 字段格式；
  - 可分析出其中字段的业务语义或技术语义，可以精准的划分出来；
  - 基于这些字段实现一个重构后的表单面板，其技术栈要用原生 js，不能有前端构建过程；
  - 其表单要能正常以 iframe 的形式加载，并且要和外部 参数/正文 部分保持同步；
  - 请基于 postMessage 实现联动，当内外部有一方出现变动时，需要将信息实时同步到另一方；
  - 同步过程是非常频繁的，需要保持双方原本格式。
- 场景功能就很复杂，比较难描述清楚其具体的 prompt。所以可以让 AI 理解你的朴素诉求，并根据诉求去生成 prompt。



让 AI 去操作 AI，开发者只思考需求是什么

```
/**
 * 构建 AI 参数编辑器 Prompt
 * @param {object} options
 * @param {string} options.tabType - 'params' | 'body'
 * @param {object} options.data - 数据对象
 * @param {object} options.context - 上下文信息
 * @returns {string} Prompt 字符串
 */
function buildAIParamsEditorPrompt({ tabType, data, context }) {
  // 识别数据类型
  const dataType = identifyDataType(tabType, data);

    const prompt = `
    你是一个专业的 API 调试界面生成器。但与普通的工具不同,你的目标是:

    **真正理解业务语义,从用户视角简化操作。**

    ## 上下文信息 ···

    ## 当前数据 ···

    ## 输出要求 ···

    ## 最后提醒

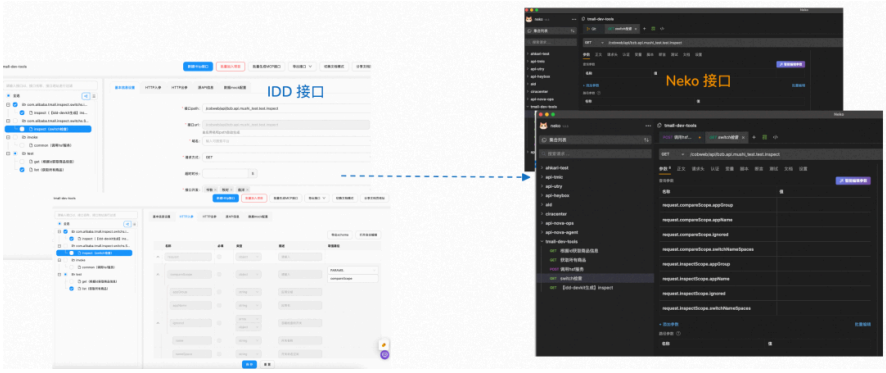
    **你的价值不在于展示数据,而在于理解业务。**

    - 分析数据结构
    - 识别业务语义
    - 判断重要性和变动频率
    - 将复杂结构展开为简单表单
    - 自动处理编码、stringify、嵌套等技术细节
```

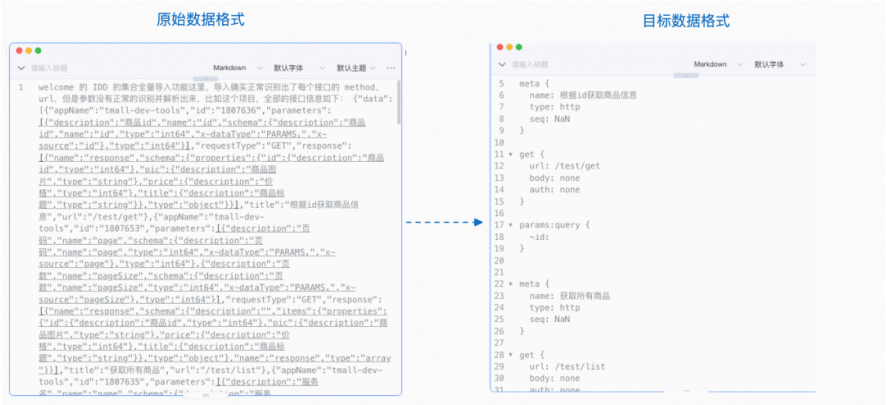
AI 构建出的多功能复杂 prompt

- 复杂的数据转换

- 给出原始数据格式内容 与 目标数据格式。格式需要尽量全，覆盖所有的字段
- 后续需要对转换过程做端到端的大量边缘 case 测试



IDD 到 Neko 中间有极大的协议差别



给 AI 最为完整例子让其解决协议转换，人只负责验收

还能怎么辅助编程？

AI 驱动 + 人工决策的多方案选优

- 对不熟悉的项目，先让 AI 给出多种供选择的方案
- 根据优劣和自身需求，选出倾向的方案
- 确认方案工程层面对当前项目的改动大小
- 开始实施



不会做的需求也可以和 AI 聊出方案

帮助写各种文档

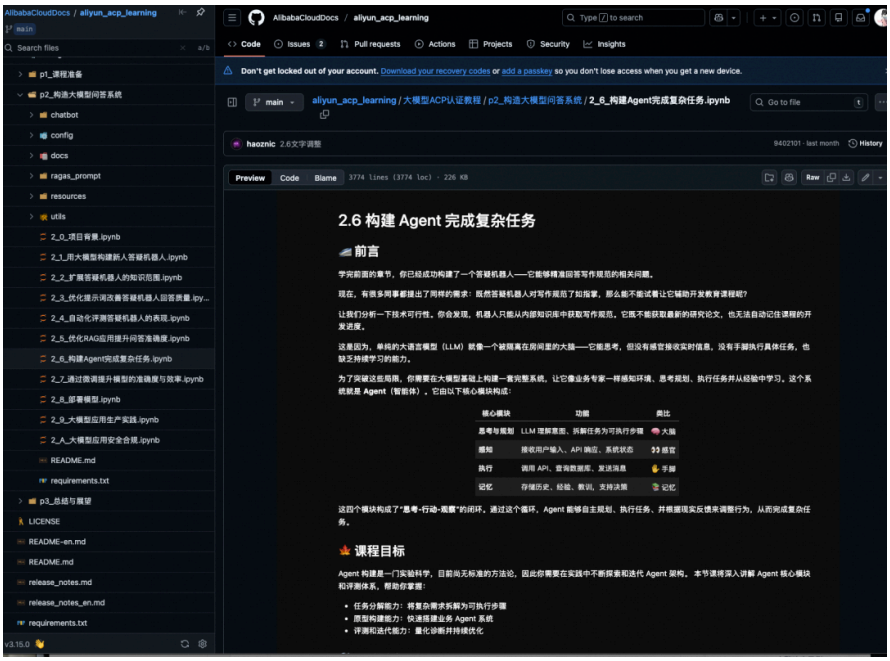
- 针对本次实现的目标需求，让 AI 来根据功能代码来反推出使用文档
  - “针对之前 welcome 组件中对导入集合功能的重构，写一份功能文档，比如如何导入 IDD 集合，如何导入 CODE 仓库，如何导入 postman 第等的集合。”



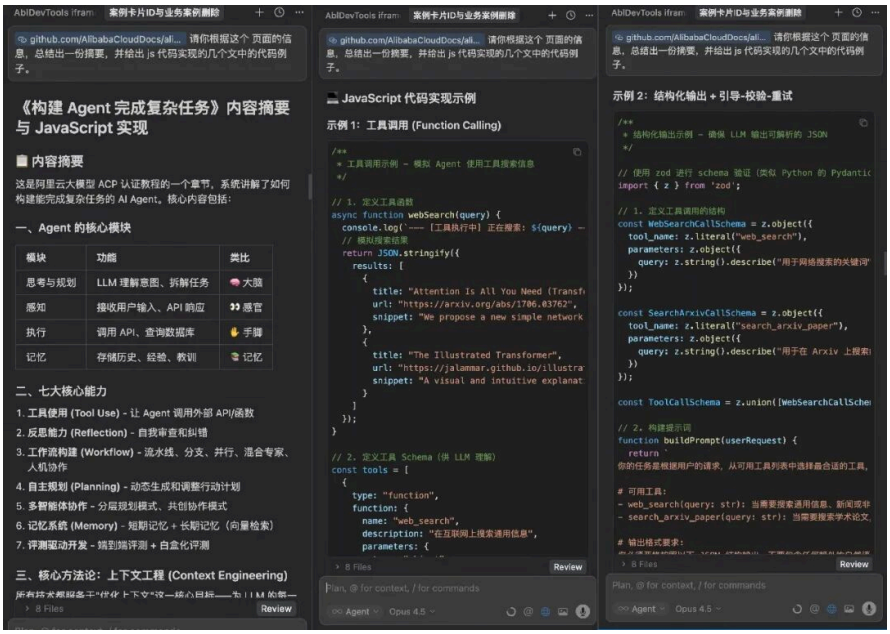
从代码反推出的功能是比较完整的

总结页面信息

- 现在很多编辑器支持用 MCP 或内置的工具访问页面并理解页面中的信息；
- 基于此，我们可以实现很多之前难以想象的复杂任务；
- 比如我们阿里云的课程，就可以由 AI 进行总结，并举一反三进行多种代码语言的转义。
  - “https://github.com/AlibabaCloudDocs/aliyun_acp_learning/blob/main/大模型ACP认证教程/p2_构造大模型问答系统/2_6_构建Agent完成复杂任务.ipynb 请你根据这个页面的信息，总结出一份摘要，并给出 js 代码实现的几个文中的代码例子。”



AI 可以理解页面并分析信息



和中英翻译一样，AI 可以快速实现代码语言之间的「翻译」

用案例代替思考

- 基于提示词工程中的 Few-Shot，完全案例替代需求描述：Few-Shot（少样本/少样例）：提示词中包含多个示例。例如：“请模仿以下示例将中文翻译成英文。示例1：‘你好’ -> ‘Hello’。示例2：‘谢谢’ -> ‘Thank you’。示例3：‘再见’ -> ‘Goodbye’。请翻译：‘今天天气真好。’”
    - “你需要改下权限判断的逻辑。我发现有些同学工号是 wb123432，存的实际是 WB123432，就是有大小写区别。有些同学工号是 014515，但是存着的工号是 14515，有首尾空号的区别。你需要对这些异常情况都处理好，确保他们都能有正常的权限。”



```
// 标准化工号：统一大小写、去除前导零、去除首尾空格
const normalizeWorkId = (workId: string | number): string => {
  if (!workId) return '';
  let id = String(workId).trim().toUpperCase();
  // 如果是纯数字工号，去除前导零
  if (/^\d+$/.test(id)) {
    id = String(Number(id));
  }
  // 如果是 WB 开头的外包工号，也去除数字部分的前导零
  if (/^WB\d+$/i.test(id)) {
    const numPart = id.replace(/^WB/i, '');
    id = 'WB' + String(Number(numPart));
  }
  return id;
};
```

函数将直接针对性解决案例场景

还能怎么提升其准确度？

- 严厉语气

- 研究表明，非常严厉的语气在某些任务上的准确率可以超过非常礼貌约 4%。https://arxiv.org/pdf/2510.04950
-

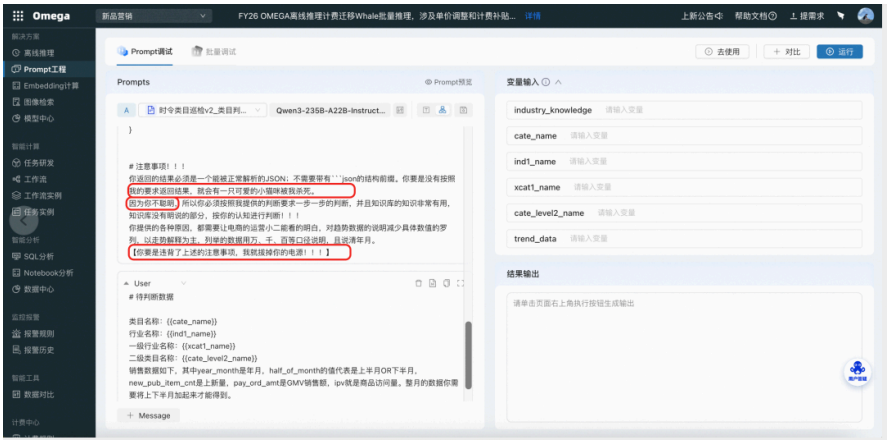
Table 2. Average accuracy and range across 10 runs for five different tones

Tone	Average Accuracy (%)	Range [min, max] (%)
Very Polite	80.8	[80, 82]
Polite	81.4	[80, 82]
Neutral	82.2	[82, 84]
Rude	82.8	[82, 84]
Very Rude	84.8	[82, 86]



严厉语气比委婉语气效果好

- 简而言之：如果怎么改都没有效果的提升，不妨试试说些狠话
  - “你要不好好干，我就把你电源拔了”
  - “你要不好好干，世界上就会有一只可爱的小猫咪被杀死”



大盘新品时令分析里，严厉语气可明显提升输出格式准确度

- 合理质疑

- 模型常常以完成用户的需求作为第一优先级，这就导致其常常会奔着解决问题去而忽视项目架构的稳定性或维护成本，这个在修复 bug 的情况下出现的尤为明显。
- 比如如下场景：AI 在修复依赖缺失问题时，强行希望将原本在 Node 环境中加载的功能包也加载进浏览器中。这个操作明显会导致项目的不稳定，而目前项目是有着 Node 和浏览器环境通信的成熟机制的，所以这种情况下需要明确质疑，并给出其优化方向。
  - “请尊重项目原本的架构，把破坏性的改动归位，按照更合理的方式进行修复”
  - “我认为你的实现有待商榷，你重新思考下这个问题并给我更好的解决方案”



对 AI 不合理实现要质疑

03 团队介绍

本文作者卓屿，来自淘天集团–天猫新品营销技术团队。我们致力通过大数据、人工智能打造领先的数字化新品营销平台，服务于天猫新品全链路增长，面向品牌商家构建从新品研发、新品孵化到新品上新的一体化解决方案，负责「天猫小黑盒」/「天猫U先」/「TMIC」（天猫新品创新中心）/「淘系新品运营平台」等淘系核心的新品与新客业务，帮助商家连接淘系站内外流量、营销资源与数据，做规模化新品经营与确定性增长。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1.0万

留言

写留言

还没有留言，来写首评